AI Usage Declaration

1. General Information

- **Course:** CS202 – Object-Oriented Programming (APCS Program)
- **Project Title:** Custom 2D Platformer (Super Mario Bros 2D Game in C++ / SFML)
- **Team:** Group 03
- **Repository:** CS202-Group03-2DGame
- **Instructor / Supervisor:**
  - PhD. DINH Ba Tien
  - MSc. TRUONG Phuoc Loc
  - MSc. HO Tuan Thanh

Team Members & Module Responsibilities

Student ID	Full Name	Primary Role & Module Assigned
25125045	**Trần Như Khải**	**Team Leader** & Hero / Items Logic (Character State/Strategy, Animation, Items)
25125057	**Trần Minh Khoa**	Core Engine & Physics (Game Loop, State Manager, AABB Collision, Camera)
25125024	**Đỗ Viết Hoàng Long**	Enemies & AI Design (Template Method Patrol AI, Enemy Hierarchy, Goomba, Koopa, Witch)
25125056	**Trần Đăng Khoa**	Tilemap, HUD & Audio (MapManager, LevelManager, HUDManager, Audio integration)

2. Declaration of AI Usage

During the design and development lifecycle of our 2D Platformer project, our team utilized Generative Artificial Intelligence tools (**Google Gemini 3.1 Pro**, **Gemini 3.6 Flash**, **ChatGPT- 5.6 Sol**, and **Gemini 1.5 Pro**) as an educational assistant and technical consultant across key areas of the project:

- **Software Architecture & Design Patterns:** Consulting standard implementations of Object-Oriented patterns (Singleton, State, Strategy, Factory Method, Template Method, and Observer) to ensure modularity, low coupling, and adherence to OOP principles.
- **Physics & Mathematics:** Understanding the theoretical concepts and mathematical formulations behind fixed-timestep game loops and Axis-Aligned Bounding Box (AABB) continuous collision detection and resolution.
- **Graphics, Sprite Generation & Asset Management:** Researching efficient 2D rendering techniques, generating sprite assets, sprite sheet parsing, and frame-based animations.
- **Build Systems & Environment Debugging:** Troubleshooting static library linking in CMake (`CMakeLists.txt`), resolving runtime DLL missing errors, and managing header inclusion dependencies.

- **Documentation & Project Management:** Generating, structuring, and editing project documentation, weekly progress reports, WBS task delegation, and Git workflows.

---

3. Statement of Responsibility

As members of Group 03, we declare that all team members take full responsibility for thoroughly reviewing, editing, refining, and testing any code, sprites, assets, or documentation generated with the assistance of AI before incorporating them into the project.

---

4. AI Interaction & Chat Log

The following list compiles all AI interactions and prompts recorded throughout the project development lifecycle:

1. **Gemini 3.1 Pro**, Google, gemini.google.com, accessed 18:30 on June 20, 2026, prompt: "*Divide the game development project tasks among 4 people. Distribute the work evenly (from UI to Core - ensuring all members get an equal share). Create a complete plan for the phases. Note: Within the OOP framework, use SFML 2.x for the render window*", used for planning the project structure, task delegation, and outlining the development phases based on OOP principles; AI provided a comprehensive Work Breakdown Structure (WBS) and a 9-week timeline, student used the plan to assign roles to team members and create the weekly report document.

2. **Gemini 3.1**, Google, gemini.google.com, accessed 07:30 on June 21, 2026, prompt: "*Guide me on creating a GitHub repo for the project: how to create the repo, invite collaborators, create branches (expected to include a dev branch and a docs branch), and basic files for the project (such as readme, weekly report template - latex, etc.)*", used for establishing the version control workflow and repository structure for the team; AI outlined step-by-step instructions for repository creation and branching strategies, student initialized the GitHub repository and invited collaborators.

3. **Gemini 3.1 Pro**, Google, gemini.google.com, accessed 22:15 on July 3, 2026, prompt: "*Provide a structural guide for building a core game engine in C++, starting with the initialization of the SFML RenderWindow and the implementation of a fixed-timestep game loop.*", used for understanding the idea for the core engine for games; AI introduced the frames to capture the states, student researches on the internet for the method, codes and runs.

4. **Gemini 3.1 Pro**, Google, gemini.google.com, accessed 22:25 on July 3, 2026, prompt: "*Resolve a runtime system error (sfml-system-d-2.dll was not found).*", used for debugging; AI suggested to put the DLLs next to the executable, then add `set(BUILD_SHARED_LIBS OFF)` into `CMakeLists.txt` to prevent the error for other members; student applies the method and checks the status.

5. **Gemini 3.1 Pro**, Google, gemini.google.com, accessed 22:30 on July 3, 2026, prompt: "*Explain the mechanics of the Singleton design pattern and demonstrate how to securely implement it to manage the core Game class without memory leaks.*", used for understanding Singleton pattern and applying this pattern into the `Game` class; AI introduced the concept and showed a demonstration for such a pattern, student researches on the internet for the method, codes and runs.

6. **Gemini 1.5 Pro**, Google, gemini.google.com, accessed 11:15 on July 4, 2026, prompt: "*Should we define the Direction struct like this or put it in a class?*", used for refactoring the architecture of the

`MoveDirection` structure from a standard struct to an `enum class` for better type safety, and designing a clean header/source directory layout for the entities component; AI provided the foundational OOP concepts and class declarations, student applied these suggestions to code the header files for the Enemy, Goomba, and Koopa classes on the development branch.

7. **Gemini**, Google, gemini.google.com, accessed 18:39 on July 4, 2026, prompt: "*Provide a structural guide for building a base Character class using the Factory Method pattern in C++, including concrete Hero products (Mario1, Mario2) with different abilities, and a HeroFactory to instantiate them.*", used for establishing the OOP architecture and entity spawning mechanics; AI generated the abstract class and factory templates, student adapted the polymorphism structure for the SFML project, debugged access modifiers, and integrated the code.

8. **Gemini 3.1 Pro**, Google, gemini.google.com, accessed 15:00 on July 11, 2026, prompt: "*Currently, Character have attributes like sprite and texture. However, for Heroes and Enemies, they have multiple states (standing, walking, jumping, dead), requiring multiple sprites for each state. Is declaring sprites in the base Character class the optimal design approach?*", used to review approach and find optimal solution; AI suggested Sprite Sheet approach, student reviewed it and accepted new approach.

9. **Gemini 3.1 Pro**, Google, gemini.google.com, accessed 16:00 on July 11, 2026, prompt: "*Help me to implement Animator class for parsing sprite sheet, storing frames and rendering correct animation based on Character's state*", used for implementing sprite animation handling; AI proposed the implementation blueprint, student reviewed codes and accepted it.

10. **Gemini 3.1 Pro**, Google, gemini.google.com, accessed 18:30 on July 17, 2026, prompt: "*How should I design the abstract State class and integrate it with a stack in my Game engine to handle seamless transitions between Menu, Playing, and Paused screens using C++?*", used to architect the core game state manager; AI suggested an Abstract Base Class and a `std::stack` implementation for state transitions, student reviewed it and accepted the architecture.

11. **Gemini 3.1 Pro**, Google, gemini.google.com, accessed 19:00 on July 17, 2026, prompt: "*When rendering SFML text using a custom .ttf font, how can I mathematically lock the UI elements to the exact center of the screen, and how should I handle window resizing to prevent the text and game coordinates from stretching?*", used to resolve UI distortion during window scaling; AI suggested using `getLocalBounds()` for dynamic centering and locking the window to a strict 1280x720 resolution, student reviewed it and accepted the static resolution approach.

12. **Gemini 3.1 Pro**, Google, gemini.google.com, accessed 23:00 on July 24, 2026, prompt: "*How do I implement the remaining lifecycle states (GameOverState, VictoryState, and TransitionState) in C++ SFML, and does a TimePerFrame of 1/60th second guarantee exact state refresh rates?*", used to finalize the state machine architecture; AI suggested the implementation blueprints for the three states and explained the fixed time-step logic, student reviewed the code and integrated the states into the engine.

13. **Gemini 3.1 Pro**, Google, gemini.google.com, accessed 23:00 on July 24, 2026, prompt: "*How do I implement HUDManager and integrate it into PlayingState in C++ SFML, and does decoupling player stats updates from the render pass optimize performance?*", used to finalize the in-game UI architecture; AI suggested the implementation blueprints for HUDManager and event-driven integration with PlayingState, student reviewed the code and integrated the HUD system into the engine.

14. **Gemini 3.1 Pro**, Google, gemini.google.com, accessed 01:00 on July 25, 2026, prompt: "*Explain the line-by-line math behind AABB collision detection in SFML, including the inverted Y-axis subtraction, the .f syntax, and how to upgrade the PhysicsEngine to prevent X-axis wall clipping.*", used to deeply understand and implement AABB physics; AI suggested a detailed mathematical breakdown of screen coordinates and a two-step movement pipeline for directional collision resolution, student reviewed the explanation, built the `PhysicsEngine` class, and tested it using dummy hitboxes.

15. **Gemini 3.6 Flash**, Google, gemini.google.com, accessed 18:00 on July 25, 2026, prompt: "*How do I structure the Enemy class header in C++ so it stays fully compatible with my teammate's Character class in feature/Hero branch without header include conflicts?*", used to check cross-teammate file dependencies; AI suggested using forward headers and relative includes, student reviewed the header paths and implemented the compatibility layer.

16. **Gemini 3.6 Flash**, Google, gemini.google.com, accessed 19:30 on July 25, 2026, prompt: "*What are the exact state transitions and gameplay mechanics for Goomba and Koopa in Super Mario Bros (stomping, squished duration, shell mode, and sliding shell velocity)?*", used to design realistic enemy behavior; AI explained the state transitions and movement parameters, student implemented the `Goomba` and `Koopa` class methods.

17. **Gemini 3.6 Flash**, Google, gemini.google.com, accessed 20:00 on July 25, 2026, prompt: "*Draw an OOP class hierarchy graph and design pattern diagram for Enemy, EnemyState, and EnemyFactory to apply Template Method and State patterns.*", used to map out object relationships before coding; AI provided an object relationship blueprint, student reviewed the diagram and wrote the C++ classes.

18. **Gemini 3.6 Flash**, Google, gemini.google.com, accessed 21:00 on July 25, 2026, prompt: "*Format my completed task breakdown and proof links for Week 07 progress report following the course markdown template.*", used to generate the weekly report documentation; AI formatted the detailed bullet points and GitHub commit URL, student reviewed and integrated it into the `week_07.md` report.

19. **Gemini 3.1 Pro**, Google, gemini.google.com, accessed 22:05 on July 25, 2026, prompt: "*How can State/Strategy Patterns be applied to the Hero for various character states?*", used to refactor the character's form logic; AI outlined the structural design and provided example interaction functions, student referenced the structure to finalize the implementation of the `Hero` class.

20. **Gemini 3.1 Pro**, Google, gemini.google.com, accessed 19:00 on July 29, 2026, prompt: "*How to implement spritesheet animation and load custom texture shapes for Koopa and Goomba enemies in a C++ SFML platformer?*", used to research sprite sheet animation rendering and texture scaling; AI provided guidance on calculating source rectangles and dynamically adjusting sprite origins, student reviewed the suggestions and implemented the sprite logic for Goomba and Koopa.

21. **Gemini 3.1 Pro**, Google, gemini.google.com, accessed 10:11 on July 30, 2026, prompt: "*Regarding the Hero design approach, should input handling logic within update(deltaTime) be placed in the Hero class, the Action State class, or the Physical Form class?*", used to determine the optimal architecture for building the Hero using State and Strategy patterns; AI proposed several architectural options along with their pros and cons, student reviewed the suggestions and selected the final implementation approach.

22. **Gemini 3.1 Pro**, Google, gemini.google.com, accessed 23:00 on Aug 4, 2026, prompt: "*How do I implement MapManager and integrate it into PlayingState in C++ SFML, and does using vertex arrays*

with viewport culling optimize tilemap rendering performance?", used to finalize the game world map architecture; AI suggested the implementation blueprints for MapManager and tile rendering optimization strategies, student reviewed the code and integrated the map system into the engine.

23. **ChatGPT- 5.6 Sol**, OpenAI, chatgpt.com, accessed on Aug 9, 2026, prompt: "Currently, the PlayingState class contains too much logic, including map loading, managing the hero and enemies, handling interactions, and processing collisions. How should I restructure this class to reduce coupling?", used to refactor the gameplay architecture; AI suggested separating responsibilities into dedicated systems/managers for map management, entity updates, physics/collision handling, and entity interactions, while keeping PlayingState primarily responsible for coordinating the gameplay flow.

24. **ChatGPT- 5.6 Sol**, OpenAI, chatgpt.com, accessed on Aug 16, 2026, prompt: "Since each map has a different playable area, regions where the hero or enemies should die when entering them, such as falling into pits, are dynamic map-specific data. In what form should the map provide this information so that it can be handled effectively now and remain extensible for future requirements?", used to design map-dependent gameplay boundaries; AI suggested representing these regions as map-defined metadata or trigger areas, such as death zones or level boundaries loaded from the tilemap, allowing gameplay systems to query map-specific environmental rules without hard-coding them into PlayingState or entity classes.

25. **ChatGPT- 5.6 Sol**, OpenAI, chatgpt.com, accessed on Aug 20, 2026, prompt: "Given the current data provided by MapManagement and the .tml/.tmj tilemap files, how can I design and handle LevelBuilder, LevelGoal, and ObjectRender?", used to determine the level-construction architecture; AI suggested a structured design in which LevelBuilder interprets map data and constructs gameplay objects, LevelGoal encapsulates level-completion conditions and destination logic, and ObjectRender handles the rendering of map-defined objects separately from level creation and gameplay logic.

---

5. Summary & Conclusion

Throughout the project development, AI was leveraged strictly in an assistive capacity to enhance software engineering quality, verify design pattern best practices, and accelerate debugging. All architectural decisions, gameplay mechanics, and final source code implementations represent the genuine engineering effort, critical evaluation, and intellectual contribution of Group 03 members.